MAUS CREPE INSIGHTS · BASELHACK 2026 · MAUS ROBOTICS CHALLENGE

From pass/fail to measurement

Maus robots bake crêpes on their own and log every run. We turned 828 of those runs into numbers that say why a crêpe failed and which setting to change next.

CrêpeFlow Team github.com/leonardoAB1/maus-crepe-insights



a real spread check: bare plate marked in yellow

Q Why do crêpe runs fail?

Nobody can tell: a failed crêpe comes with a verdict, not a reason

430 runs that started a crêpe



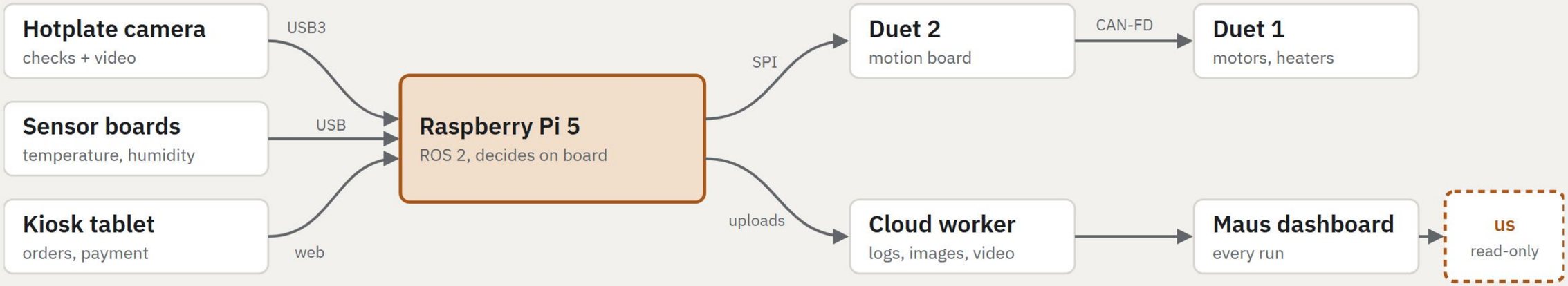
31 with a reason: 12 spread, 9 fold 2, 6 plate checks, 4 fold 1

79% of failed runs cannot be explained from the data. Where a check exists, it is still only pass or fail.

Source: documentation/system/crepe-stages.md, runs table (828 runs, 3 robots)

Q What does the robot see, and what does it decide on its own?

One robot, one Pi, a camera over the hotplate



A run, about 3.5 minutes; the crêpe is served after about 2.5. **Orange** = a camera check with an image.

Prep
prepare, oil
plate checks

Bake
spread batter, cook, topping
spread

Serve
fold, push off, handover
fold 1 · fold 2 · push-off

Reset
wipe, discard, plate load
outcome decided here

Source: documentation/system/system.md, documentation/system/crepe-stages.md

Q Before any vision model: can plain data, control theory and experiments explain the failures?

Partly: data explains a lot, but most failures leave no data behind

Data analysis

QUESTION

Which settings and conditions go with failure?

History analysis over **401** hot full cycles and the robot's settings.

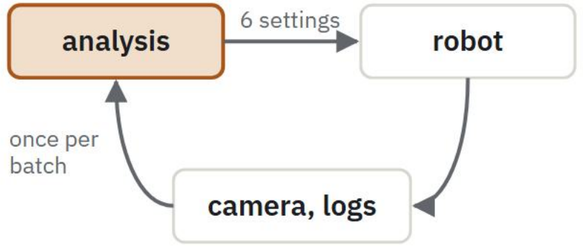
ANSWER

Failures follow the robot, not the spread: fold failure is 21% on maus-quin and 61% on maus-pixi.

Control theory

QUESTION

What is the robot, seen as a system?



ANSWER

A run-to-run loop that a person closes today by trial and error.

Experiments

QUESTION

Can the history tell cause from coincidence?

No: settings were changed together, a few runs at a time.

ANSWER

A Design of Experiments (DoE): 19 runs that vary six settings independently.

Numbers beat pixels where numbers exist

On fold 2, a simple regression on the robot's own overlay scores reaches **0.88**, against **0.67** for an image model.

But most failures have no numbers

116 of 147 failed runs carry no check data. That is where vision comes in.

Source: documentation/analysis/analysis.md, doe.md, fold-risk.md, documentation/status.md

Q What goes with failure in the run history?

The data already explains a lot: failures follow the robot, cancels follow the wait

Cancels happen while customers wait

34 of 66 cancels come during cook, the longest stage (median 47 s). A chocolate topping adds another 61 s.

Change: show progress, or start the next order sooner

Fold failures follow the robot

Once the robot is accounted for, a bad spread does not predict a bad fold (odds ratio 1.67, 95% interval 0.50 to 5.53).

Change: compare maus-pixi's setup with maus-quin's

One lead worth a test

Bare plate at spread time predicts residue at fold time: odds ratio **2.47** per SD (1.12 to 5.56).

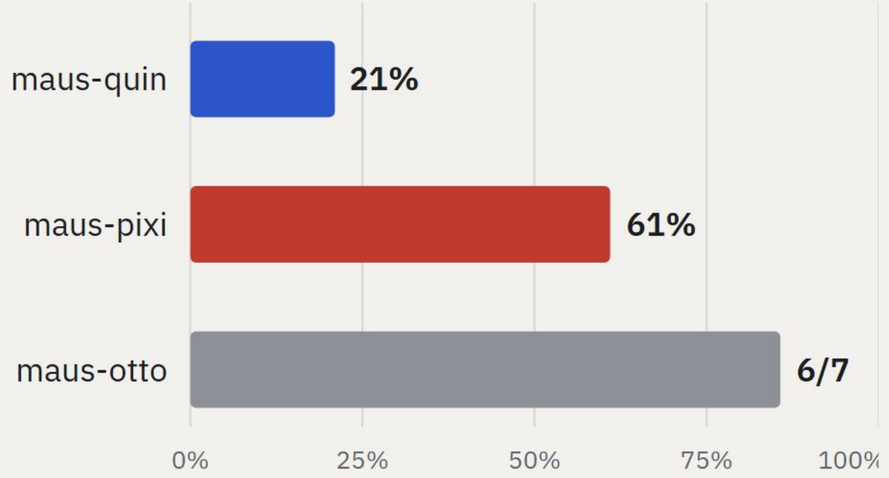
Test: include it in the next batch

Most failures are unrecorded

116 of 147 failed runs carry no check data, so their cause is unknowable today.

Change: record every check on every run

Fold-failure rate per robot



maus-otto has only 7 such runs. Robot and time period coincide in this data, so the batch on the next slide separates them.

Source: documentation/system/crepe-stages.md, documentation/analysis/fold-risk.md

Q Which recipe settings should Maus change, and which way?

The history points at two knobs: lower fan inset, raise fan power

SETTING	BATCH RANGE · ● NOW	WHAT THE RUN HISTORY SAYS
fan inset (mm)	5 — 20	**Lower** — more fold 1 passes; it now sits at the top of its range
fan power	0.85 — 1	**Raise** — bigger spreads that pass; it now sits at the bottom
y center	−46 — −42	**Lower** — better fold 1 and fold 2; little room left
z scrape start	38.9 — 41.5	**Lower, unsure** — fold 1 improves, but single-change tests disagree
inner flowrate (g/s)	12 — 14	**Hold at 13** — 0 of 9 runs succeeded at any other value
outer fan tail (deg)	5 — 25	**No signal yet** — no clear effect; 5 deg succeeded once in 7 runs

1
Confirm whether y center and z scrape start may change per run. If not, the batch shrinks to 4 settings.

2
Quick test the two clearest moves: fan power up from 0.85, fan inset down from 20. Keep inner flowrate at 13.

3
Run the 19-run DoE sheet on one robot, plain topping, shuffled order, with check stills saved on every run.

4
Send back the run ids. The analysis says which knob, which way, and suggests the next 4 runs.

Source: history analysis over 401 hot full cycles (documentation/analysis/analysis.md); now = median of the last 30 runs; DoE: documentation/analysis/doe.md

Q How do we prove which setting causes what?

A Definitive Screening Design turns hints into causes in 19 runs

SETTING	LOW	MID	HIGH	HISTORY SAYS
fan power	0.85	0.925	1.0	raise: larger spread
outer fan tail (deg)	5	15	25	
inner flowrate (g/s)	12	13	14	13 only ever worked
fan inset (mm)	5	12.5	20	lower: better fold 1
y center	−46	−44	−42	lower: better folds
z scrape start	38.9	40.2	41.5	

1. Maus runs a ready sheet: 16 design runs plus 3 centre runs, in shuffled order.
2. Each setting's effect is measured on its own, including its sweet spot.
3. Our pipeline scores every run, then the fit says which knob to turn and which way.
4. Score every run live on the Pi 5, at about 0.15 s per frame.

Why a Definitive Screening Design (DSD)

19 runs, not 729

Testing every combination of 6 settings at 3 levels takes $3^6 = 729$ runs.

Each setting on its own

No effect is mixed up with another setting or with a curve: their correlation in the design is 0.

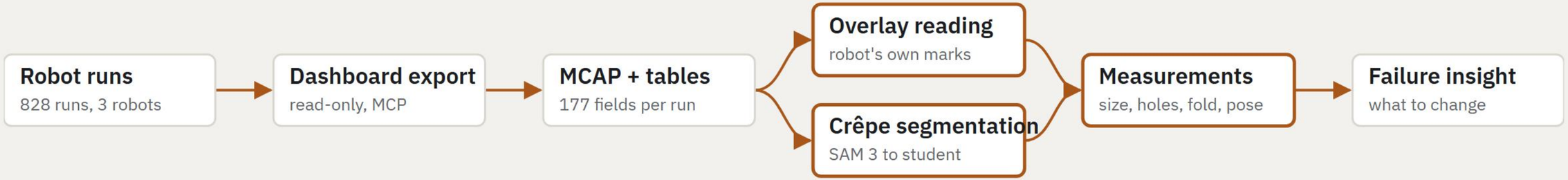
Sweet spots visible

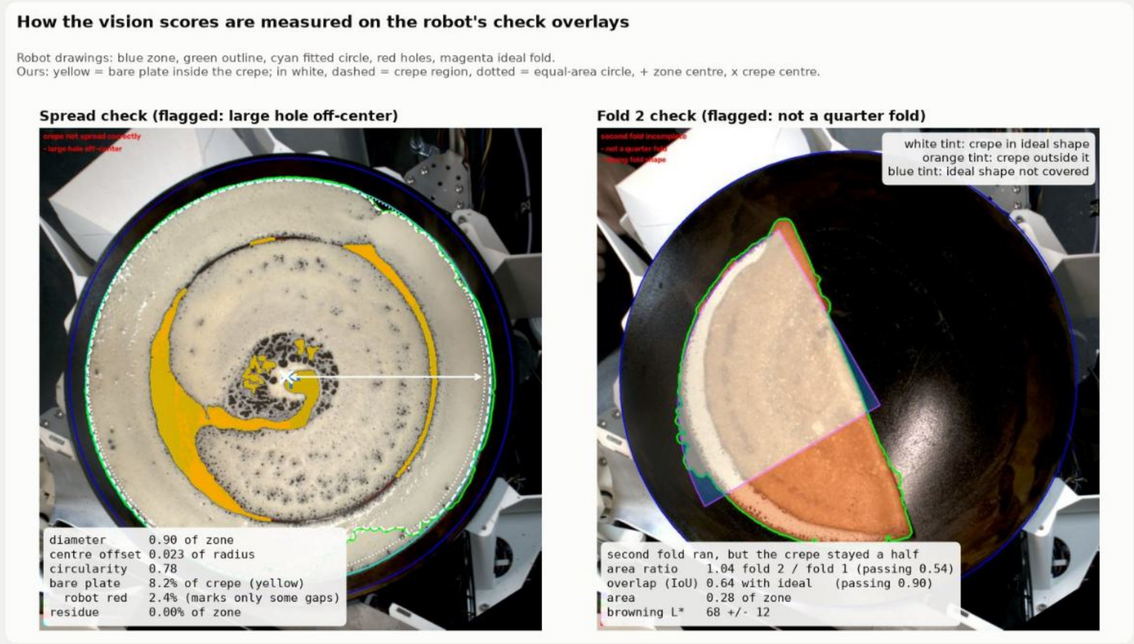
Three levels show whether the best value lies inside the range; 3 centre runs measure run-to-run noise.

Source: documentation/analysis/doe.md, code/maus_export/doe.py

Q How do we measure the runs that recorded nothing?

Where the numbers run out, we gave the robot better eyes





The robot already draws its analysis

- analysis zone
- crêpe outline
- holes
- ideal fold shape

We read those marks back per run: diameter, centre offset, circularity, bare plate, residue, fold overlap, fold 2 / fold 1 area ratio and browning. A distilled segmentation model gives the same view on live video.

Q Does the vision work, and is it fast enough?

The eyes work: mask, fold and pose, running on a Pi

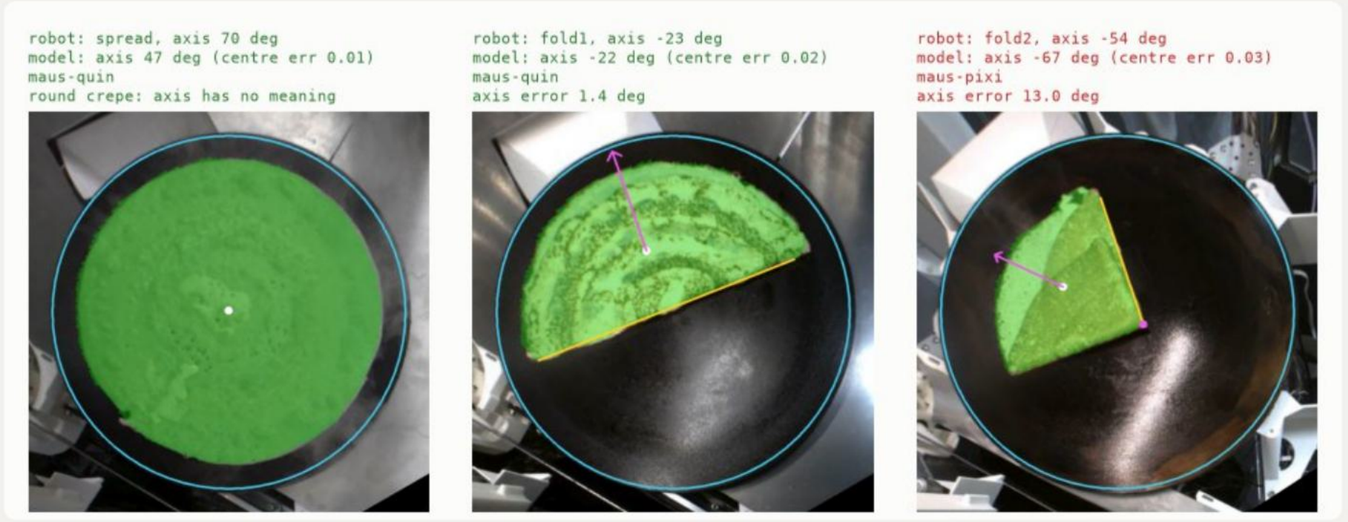
0.966
mask IoU vs SAM 3
225 held-out frames, 57 runs

0.944
median IoU vs human masks
62 frames; two humans agree at 0.933

99.5%
fold state matches the robot
991 checks; classical pipeline 88.5%

1.1°
fold axis error, median
centre within 0.023 plate radii (fold 1)

0.15 s
per frame on a Raspberry Pi 5
measured end to end; under the 300 ms target

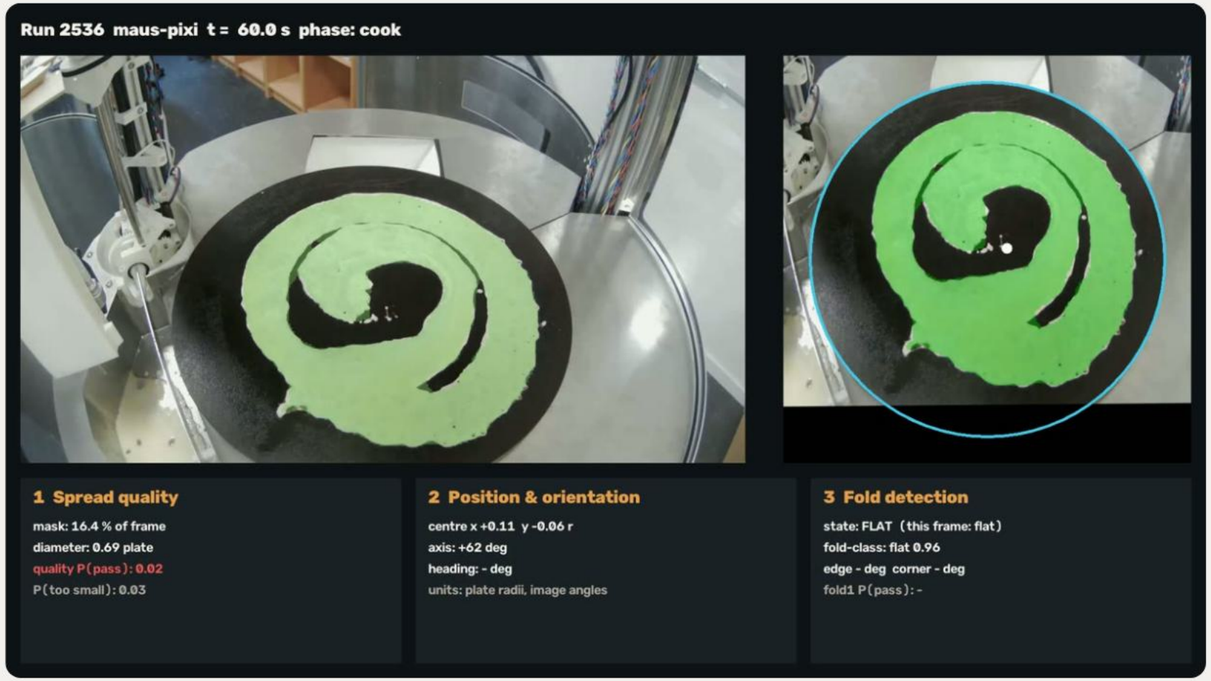


References are SAM 3, the robot's own overlays and a five-run human pilot on one robot. Right tile: a fold 2 miss of 13°.

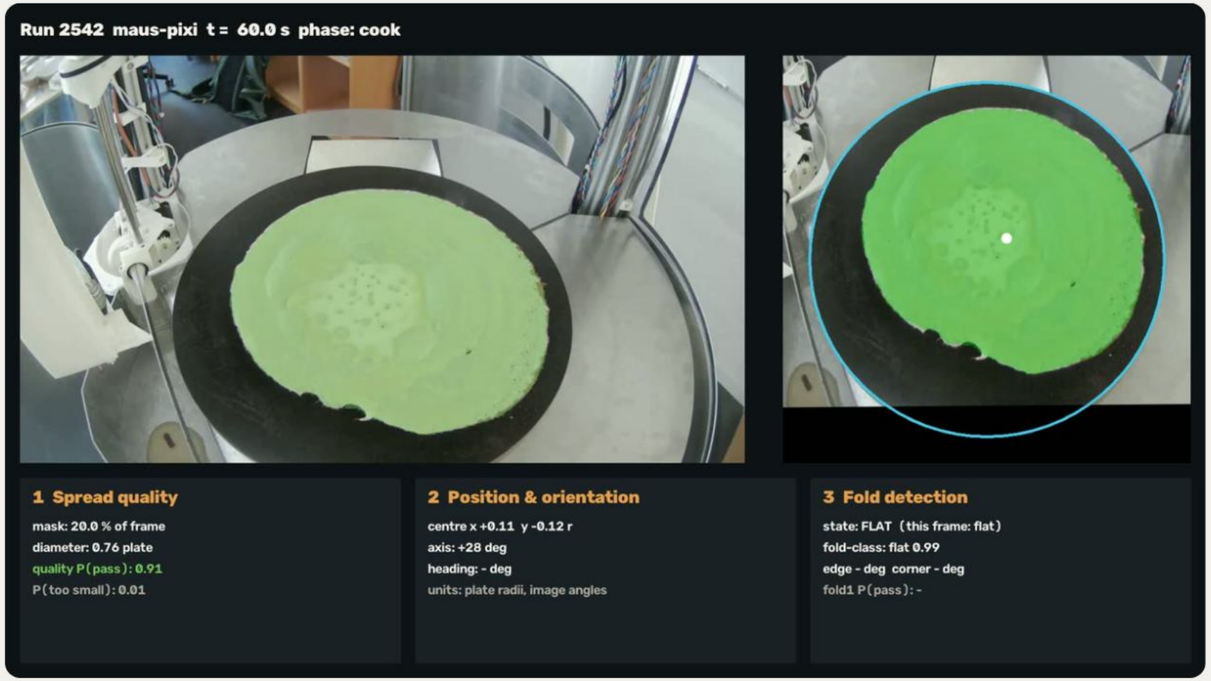
Source: code/models/crepe-student/model.json, documentation/training/crepe-student.md

Q What does it look like on a real run?

One minute into two runs: a broken spread scores 0.02, a good one 0.91



flagged Run 2536: the batter spirals and leaves holes; spread quality P(pass) 0.02 at 60 s



good Run 2542: a full, round spread; spread quality P(pass) 0.91 at 60 s

Every frame shows the three brief tasks: spread quality, position and orientation, and fold detection, next to the top-down view.

Q Does it run on robot hardware today?

Yes: on a Raspberry Pi 5, about 0.15 s per frame

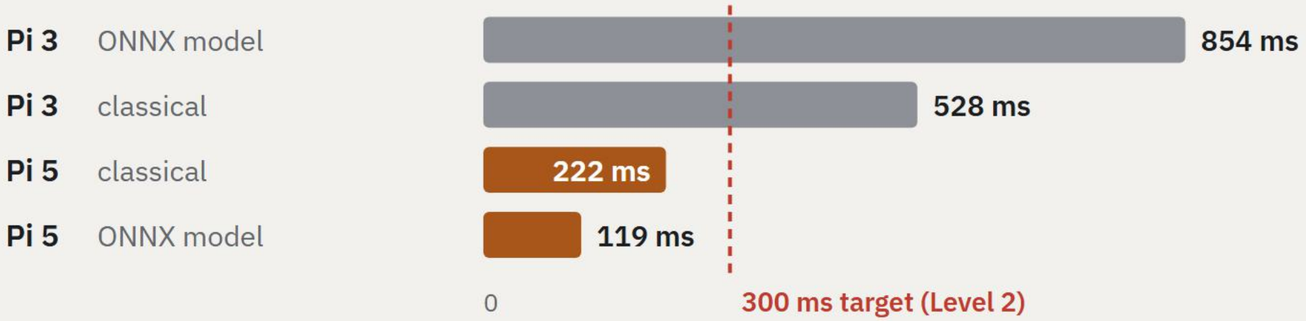
192.168.22.131

Open it on the venue Wi-Fi **memox.world**. Our Pi 3 runs the same site at 192.168.23.119.

Every run's video with the crêpe mask and fold state, and each robot check next to our verdict and a "disagrees with robot" marker.

scan on the venue Wi-Fi

Analysis time per frame on both boards, median



8 of 8

fold 1 and fold 2 checks agree with the robot (Pi 3 demo, 5 runs)

12 of 13

crêpes found; none on the 5 empty plates

End to end, a whole run on the Pi 5 takes 0.15 s per frame (4,510 frames, 3 runs), including reading the video. The bars show the analysis of each frame: correction, segmentation, size, pose, fold state, drawing. Spread agrees on 2 of 4 checks.

APPENDIX

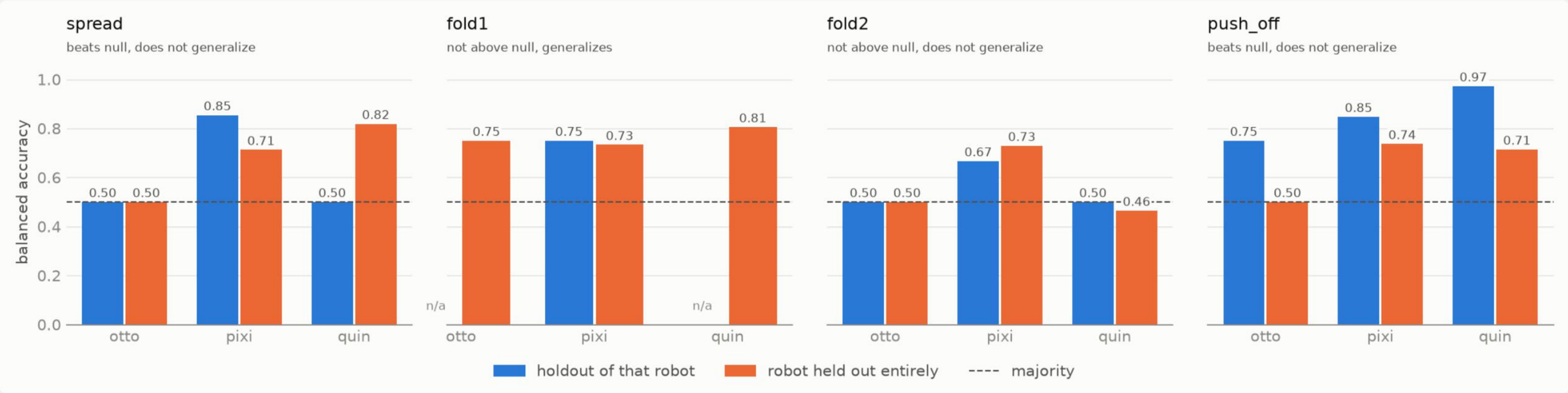
The brief scorecard, next steps, and how to check our work

Every brief item has a working answer

BRIEF ITEM	STATUS	WHAT WE DELIVERED	NEXT
Task 1: spread quality	delivered	Crêpe mask at IoU 0.966 vs SAM 3 and 0.944 vs human masks; diameter, circularity, holes and bare plate measured per run	hole class in our own mask
Task 2: centre and angle	delivered	Centre within 0.023 plate radii, fold axis within 1.1° (median, fold 1); push-off angle read from 248 of 257 overlays	millimetres in robot coordinates
Task 3: fold detection	delivered	Flat, half and quarter at 99.5% agreement on 991 checks; fold edge found on every passed fold 1	triangle fold
Level 1: works on Maus data	met	99.5% fold-state agreement with the robot's own checks; on the Pi, fold checks agree 8 of 8	more robots in the holdout
Level 2: real time on a Pi 5	met on a Pi 5	Measured on a Pi 5: 0.15 s per frame end to end (119 ms of analysis) for mask, size, pose and fold state, over 4,510 video frames from 3 runs	time the pass/fail check heads per frame

Source: documentation/status.md, model cards in code/models/, Pi 3 and Pi 5 measurements

What comes next



- Train on the 186 phase labels and 113 masks we already collected, so models learn from people, not only robot verdicts.
- Record check stills on every run: today 138 of 828 runs have vision metrics.
- Time the pass/fail check heads per frame on the Pi 5; the per-frame pipeline already runs at 0.15 s.
- Train across more robots, so every check holds up on a robot the model has never seen.
- Add a material and hole mask, so the deviation check runs on real predictions.
- Rerun all numbers with Maus: the code and tests are public, the run data stays private.

Check it yourself

```
git clone https://github.com/leonardoAB1/maus-crepe-insights
cd maus-crepe-insights/code
pixi install && pixi run test && pixi run lint
```

The test suite runs offline, without Maus data. Every number in this deck has a source in `documentation/status.md` and the model cards in `code/models/`.

Licence: PolyForm Noncommercial 1.0.0.